Clustering Pitch Types with kmeans: 2026 Pitches

Goal

The goal of this project was to see whether a model could sort pitches into the right pitch types using only their measurements. If a model only knew a pitch's velocity, movement, and spin, and was never told what anyone called the pitch, could it group pitches the same way Statcast labels them? I used kmeans clustering in R, a method that sorts data points into a chosen number of groups (called k) based on how similar their measurements are, and tested which combination of measurements did the best job. The four predictors were velocity, induced vertical break, horizontal break, and spin rate. The pitch type labels were hidden from the model and only used afterward to score how well it did.

Methodology

1. Every pitch from the 2026 regular season was pulled from Baseball Savant using the statcast_search function from the baseballr package and saved as an RDS file. The pitches were then loaded into a DuckDB database with dbWriteTable, so the cleaning and summarizing could be done in SQL. The duckdb, DBI, dplyr, cluster, and ggplot2 libraries were loaded in R.
2. The pitches were cleaned in SQL and saved as a view called pitches_clean.
    a. Pitches with no pitch type or no tracking data were removed.
    b. Pitch types that aren't normal pitches or are too rare to use were removed: eephus pitches, pitches labeled "Other," pitchouts, unknown pitches, and knuckleballs. Most eephus and "Other" pitches come from position players pitching, and knuckleballs were only thrown by a handful of pitchers and don't follow normal spin and movement patterns.
    c. Vertical and horizontal break were converted from feet to inches.
    d. Horizontal break was mirrored for left handed pitchers so arm side movement is positive for everyone. Without this, a lefty's changeup and a righty's changeup would land on opposite sides of the chart even though they're the same pitch.
3. A second SQL query grouped the pitches into one row per pitcher per pitch type, averaging velocity, spin rate, vertical break, and horizontal break, and saved it as a table called pitch_summary.
4. The clustering data was pulled from pitch_summary with a third SQL query.
    a. Only rows with at least 50 pitches thrown and a recorded spin rate were kept, so each row's averages were based on a reasonable sample.
    b. CASE statements created two labels for each row. The first was the pitch family (Fastball, Breaking, or Offspeed). The second was a pitch label with eight groups: four seamer, sinker, cutter, changeup, splitter, curveball, slider, and sweeper. Rare pitch types were folded into the closest group the same way Baseball Savant groups them: forkballs with splitters, knuckle curves and slow curves with curveballs, and slurves with sliders.

    c. dbGetQuery read the results into R as pt. nrow() and table() were run to confirm the row count and find the share of the largest group, which works as a baseline: the accuracy a model would get without learning anything at all.
5. The four predictors were standardized with scale(). They are on very different scales (spin rate is in the thousands while break is in inches), so without scaling, spin rate would have dominated the distance calculations.
6. A function called score_clusters was written to score each clustering. table() compared the clusters to the true labels, the count of the most common label in each cluster was taken with apply() and max(), and those counts were summed and divided by the total number of rows. This is sometimes called cluster purity.
7. Every possible combination of the four predictors was tested with k = 3, meaning kmeans was asked to find three groups to match the three pitch families.
    a. combn() built all 15 combinations: four single predictors, six pairs, four groups of three, and all four together.
    b. For each combination, set.seed(6501) was run right before kmeans so the random starting points would be the same every time, and nstart = 25 ran each model from 25 different starting points and kept the best one.
    c. Each result was scored against the pitch family and saved in a results table.
8. The best combination was refit, and table() was used to compare its clusters to both the pitch family and the specific pitch label. This showed exactly which pitch types landed in the wrong family's cluster.
9. To suggest a value of k, kmeans was run for every k from 2 to 10 using all four predictors.
    a. tot.withinss was recorded for each k for the elbow method.
    b. The average silhouette width was also recorded using the silhouette function from the cluster package. It measures how cleanly separated the clusters are, from about 0 (overlapping) to 1 (very distinct).
    c. Both measures were plotted against k.
10. kmeans was run again with k = 7 and k = 8 to see if it could recover the specific pitch types.
    a. For k = 7, sweepers were folded into sliders so the labels matched Baseball Savant's slider group.
    b. For k = 8, sweepers were kept separate to test whether the model would find them as their own pitch.
    c. Both were scored with score_clusters, and table() was used to see which pitch types landed in each cluster.
11. Each of the k = 8 clusters was described in real units using group_by and summarise from dplyr, including the most common pitch label in the cluster, its share of the cluster, and the average velocity, vertical break, horizontal break, and spin rate. Each cluster was then given a name based on those averages.
12. Three charts were made with ggplot2 and saved with ggsave, using Baseball Savant's pitch colors.

a. A movement chart (horizontal break vs vertical break) colored by the Statcast label.
b. The same movement chart colored by kmeans cluster, so the two could be compared side by side.
c. A velocity vs spin rate chart colored by kmeans cluster.
d. On every chart, each small dot is one pitcher's pitch type and each large outlined circle is the average of its group.

Results

Baseline. After cleaning, there were 2,574 rows, each one a pitcher's pitch type with at least 50 thrown. Fastballs made up 1,265 of them, or 49.1%. Since the scoring method credits each cluster with its most common label, a model that lumped every pitch into one group would still score 49.1% for the families, so that's the floor any model has to beat. For the eight specific pitch types, four seamers were the biggest group at 597 rows, so the floor there is 23.2%.

```
> # Data check 1: total rows, plus how many rows fall under each label
> # (the labels are only used for scoring, never in the clustering itself)
> nrow(pt)
[1] 2574
> table(pt$family)

Breaking Fastball Offspeed
     903     1265      406
> table(pt$pitch_label)

   Changeup   Curveball      Cutter Four Seamer      Sinker      Slider    Splitter
        324         248         242         597         426         399          82
    Sweeper
        256
```

Predictor combinations (k = 3). All four predictors together had the best accuracy at 93.6%. Spin rate was part of every one of the top five combinations, and velocity plus spin rate alone reached 91.3% with just two predictors. Movement on its own was much weaker, with vertical and horizontal break together only reaching 80.1%.

Accuracy for every predictor combination at k = 3 (velo = velocity, ivb = induced vertical break, hb = horizontal break, spin = spin rate):

Predictor(s)	Accuracy
velo + ivb + hb + spin	93.6%
velo + ivb + spin	92.9%
velo + spin	91.3%
velo + hb + spin	90.8%
ivb + hb + spin	90.7%
velo + hb	89.5%
ivb + spin	88.5%
hb + spin	85.3%
ivb + hb	80.1%

Predictor(s)	Accuracy
velo + ivb + hb	80.0%
velo + ivb	76.3%
velo	74.9%
spin	72.6%
hb	69.7%
ivb	68.9%

Where the best model went wrong. The model using all four predictors missed 165 of the 2,574 rows. Comparing its clusters to the pitch labels showed that most of those misses came from one pitch. 100 of the 242 cutters (41%) landed outside the fastball cluster, almost all of them in the breaking ball cluster, which accounts for about 61% of all the misses. Without cutters, the model would have been right about 97.2% of the time. The other misses were mostly 24 sinkers grouped with the offspeed pitches, 16 changeups grouped with the fastballs, and 13 sliders grouped with the fastballs. Curveballs and sweepers were placed correctly every time.

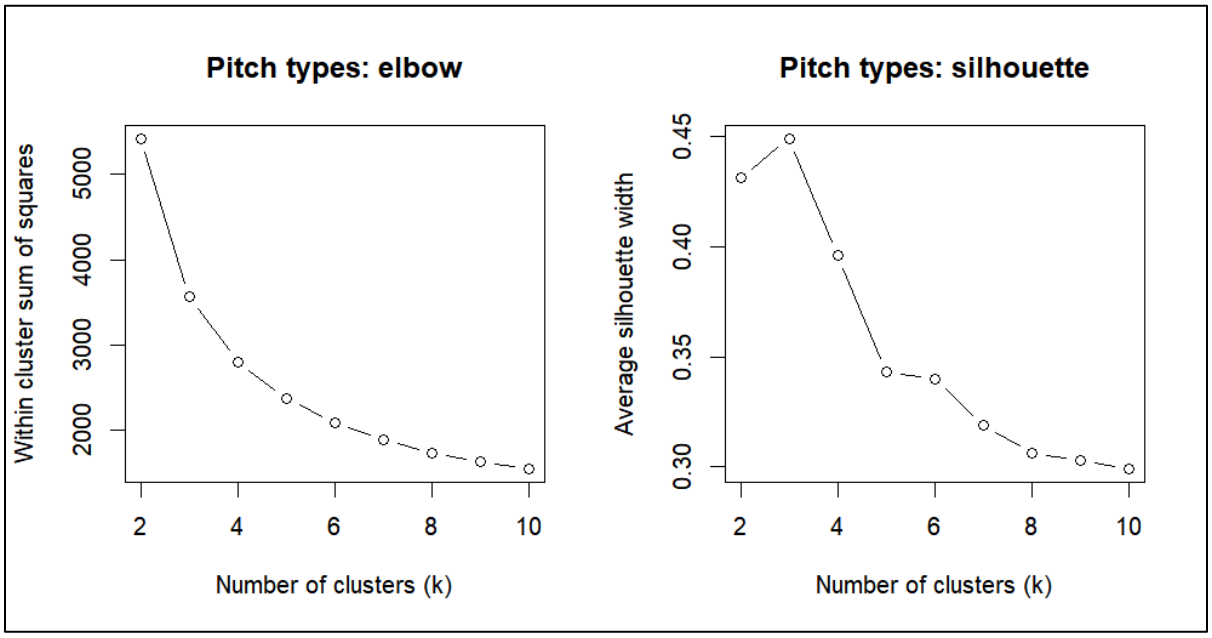
```
> # Which family each cluster mostly matched (the majority family in each cluster)
> table(cluster = km3_best$cluster, family = pt$family)
       family
cluster Breaking Fastball Offspeed
      1        4       28      385
      2       13     1138       18
      3      886       99        3
>
> # Same clusters, broken out by specific pitch type
> # This shows exactly which pitch types are landing in the "wrong" family's cluster
> table(cluster = km3_best$cluster, pitch = pt$pitch_label)
       pitch
cluster Changeup Curveball Cutter Four Seamer Sinker Slider Splitter Sweeper
      1      305         0      2           2     24      4       80        0
      2       16         0    142         595    401     13        2        0
      3        3       248     98           0      1    382        0      256
```

Share of each pitch type placed in its own family's cluster:

Pitch type	Rows	In its family's cluster	Share
Four Seamer	597	595	99.7%
Sinker	426	401	94.1%
Cutter	242	142	58.7%
Changeup	324	305	94.1%
Splitter	82	80	97.6%
Curveball	248	248	100%
Slider	399	382	95.7%
Sweeper	256	256	100%

Choosing k. The elbow plot bends most sharply at k = 3. The within cluster sum of squares dropped by 1,849.9 going from 2 to 3 clusters, but only 759.3 going from 3 to 4, and the drops kept shrinking after that. The silhouette width also peaked at k = 3 at 0.449, compared to 0.431 at k = 2 and 0.396 at k = 4. Past that, it kept falling, to 0.319 at k = 7 and 0.306 at k = 8. So when the model is left to decide, the data naturally falls into three groups, which line up with the three pitch families. My suggested value of k is 3.

```
> # How much within_ss dropped from the previous k (where this levels off is the elbow)
> k_check$ss_drop <- c(NA, round(-diff(k_check$within_ss), 1))
> print(k_check)
   k within_ss avg_silhouette ss_drop
1  2    5415.0          0.431      NA
2  3    3565.1          0.449  1849.9
3  4    2805.8          0.396   759.3
4  5    2377.6          0.343   428.2
5  6    2087.0          0.340   290.6
6  7    1892.5          0.319   194.5
7  8    1730.3          0.306   162.2
8  9    1633.9          0.303    96.4
9 10    1549.8          0.299    84.1
```



Specific pitch types (k = 7 and k = 8). With k = 7, the model scored 80.6%, and with k = 8 it scored 82.8%, both far above the 23.2% floor. But the groups it found didn't match Baseball Savant's labels one for one. In both runs, sweepers got their own cluster (228 of 256 at k = 7 and 216 of 256 at k = 8). Changeups were split almost evenly across two clusters (161 and 160 at k = 7, 160 and 160 at k = 8), and splitters never got a cluster of their own, with 69 of the 82 landing in one of the changeup clusters. Cutters and sliders shared one large cluster at k = 7, and at k = 8 cutters got their own cluster but it still pulled in 82 sliders.

```
> # ---- Step 4: can kmeans recover the specific pitch types? (k = 7 and k = 8) ----
>
> # For k = 7, fold sweepers into sliders so the labels match Savant's slider group
> pt$label7 <- ifelse(pt$pitch_label == "Sweeper", "Slider", pt$pitch_label)
>
> # k = 7 using all 4 predictors, scored against the 7 pitch groups
> # (floor is about 23%, the share of four seamers)
> set.seed(6501)
> km7 <- kmeans(pt_scaled[, predictors], centers = 7, nstart = 25)
> round(score_clusters(km7$cluster, pt$label7) * 100, 1)
[1] 80.6
>
> # Which pitch types landed in each of the 7 clusters
> table(cluster = km7$cluster, pitch = pt$pitch_label)
       pitch
cluster Changeup Curveball Cutter Four Seamer Sinker Slider Splitter Sweeper
      1        1         14    212           1      0    319        0       19
      2        0          0     29         548     32      0        0        0
      3      161          0      0           5     38      0       11        0
      4        0        221      0           0      1     15        0        9
      5        0         13      1           0      0     64        0      228
      6        2          0      0          43    355      0        2        0
      7      160          0      0           0      0      1       69        0
```

```
> # k = 8 using all 4 predictors, scored against the 8 pitch labels (sweeper separate)
> set.seed(6501)
> km8 <- kmeans(pt_scaled[, predictors], centers = 8, nstart = 25)
> round(score_clusters(km8$cluster, pt$pitch_label) * 100, 1)
[1] 82.8
>
> # Which pitch types landed in each of the 8 clusters
> table(cluster = km8$cluster, pitch = pt$pitch_label)
       pitch
cluster Changeup Curveball Cutter Four Seamer Sinker Slider Splitter Sweeper
      1      160          0      0           5     37      0       11        0
      2        2          0      0          41    353      0        2        0
      3        2         28     21           0      0    278        0       31
      4      160          0      0           0      0      1       69        0
      5        0         11      0           0      0     31        0      216
      6        0          0    212           9      0     82        0        1
      7        0          0      9         542     35      0        0        0
      8        0        209      0           0      1      7        0        8
```

Naming the k = 8 clusters. Describing each cluster in real units made it possible to name them. The two changeup clusters had almost the same velocity (87.2 vs 85.9 mph), but one averaged 1,973 rpm of spin and the other only 1,410 rpm. The higher spin one has the same arm side run as the sinker cluster, just about 7 mph slower, and the lower spin one is where most of the splitters landed.

```
> print(k8_profile)
  cluster rows   top_label top_share velo   ivb    hb spin
1       1  213    Changeup      75.1 87.2   5.0  15.2 1973
2       2  398      Sinker      88.7 94.4   8.2  15.1 2217
3       3  360      Slider      77.2 86.0   0.2  -4.2 2432
4       4  230    Changeup      69.6 85.9   3.0  11.8 1410
5       5  258     Sweeper      83.7 82.0   1.3 -13.7 2598
6       6  304      Cutter      69.7 89.6   7.4  -2.1 2369
7       7  586 Four Seamer      92.5 94.8  15.7   7.6 2316
8       8  225   Curveball      92.9 80.1 -10.9  -9.0 2586
```

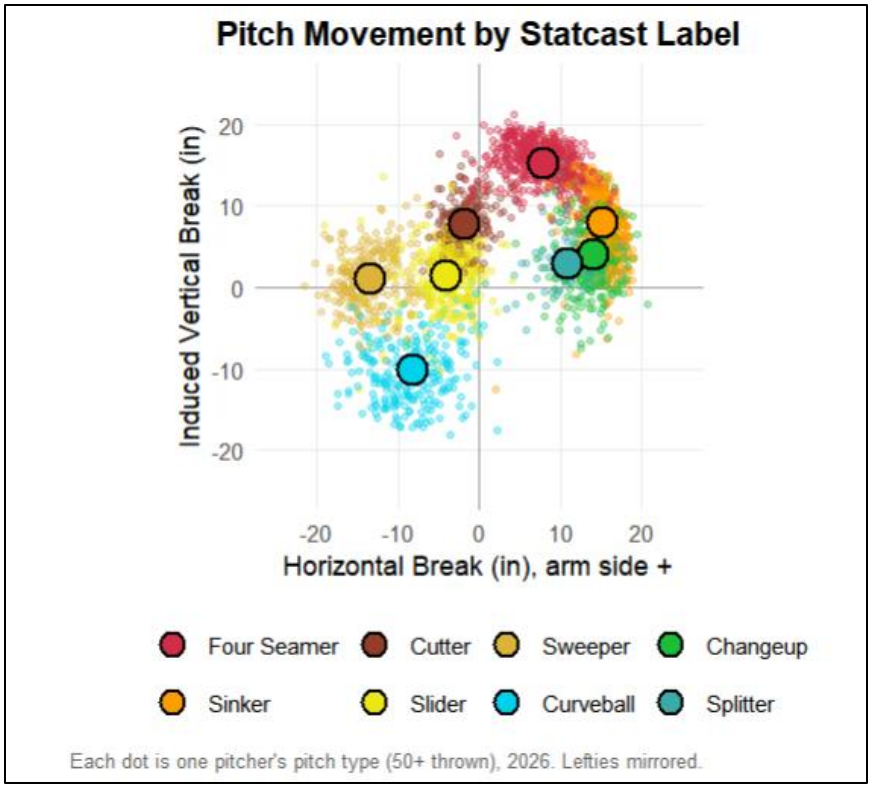
The eight clusters, named by their averages (velocity in mph, IVB = induced vertical break and HB = horizontal break in inches with arm side positive, spin in rpm):

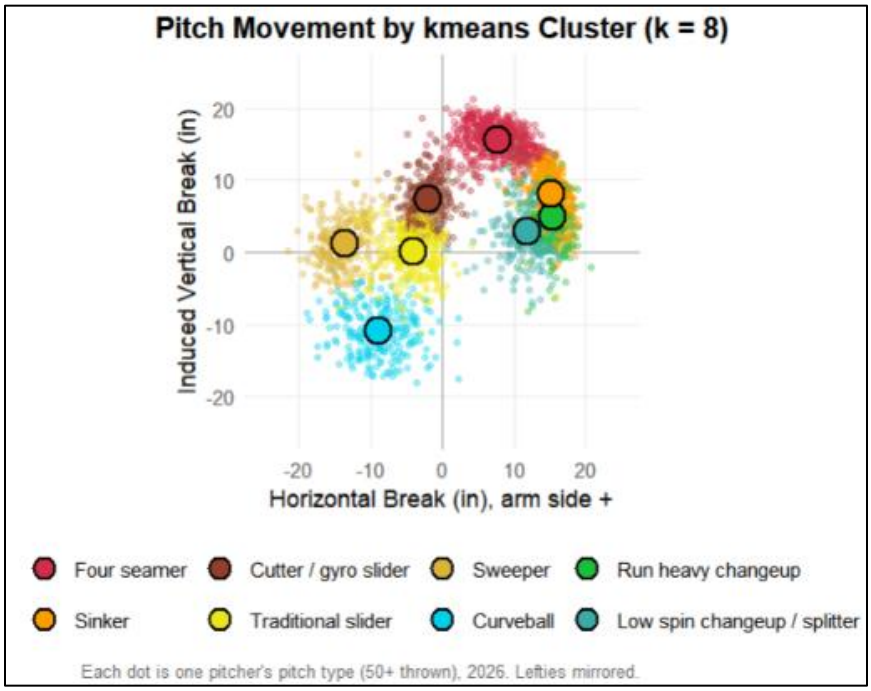
Cluster	Name	Most common label	Velo	IVB	HB	Spin
7	Four seamer	Four Seamer (92.5%)	94.8	15.7	7.6	2,316
2	Sinker	Sinker (88.7%)	94.4	8.2	15.1	2,217
6	Cutter / gyro slider	Cutter (69.7%)	89.6	7.4	-2.1	2,369
1	Run heavy changeup	Changeup (75.1%)	87.2	5.0	15.2	1,973
3	Traditional slider	Slider (77.2%)	86.0	0.2	-4.2	2,432
4	Low spin changeup / splitter	Changeup (69.6%)	85.9	3.0	11.8	1,410
5	Sweeper	Sweeper (83.7%)	82.0	1.3	-13.7	2,598
8	Curveball	Curveball (92.9%)	80.1	-10.9	-9.0	2,586

Summary of accuracy:

k	Scored against	Accuracy	Floor
3	Pitch family	93.6%	49.1%
7	7 pitch groups (sweeper in slider)	80.6%	23.2%
8	8 pitch labels	82.8%	23.2%

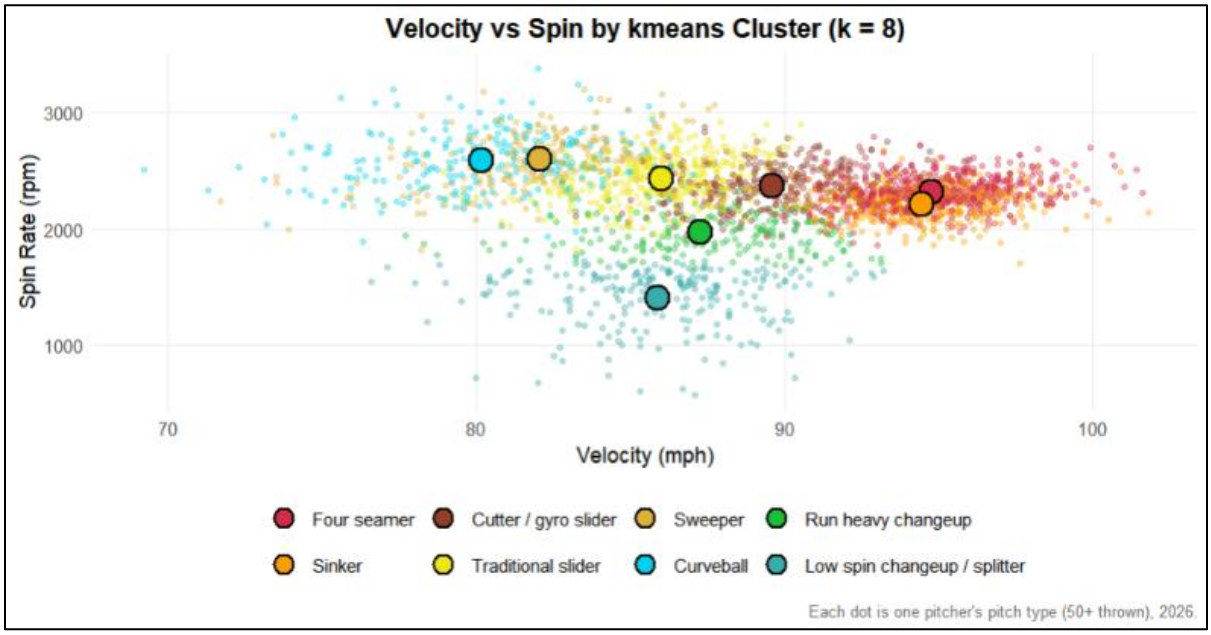
Movement charts. The two movement charts below show the same pitches, first colored by what Statcast calls them and then colored by kmeans cluster. The cluster chart mirrors the Statcast chart closely, which shows kmeans recovered most of the pitch types on its own. The main differences are in the middle, where cutters and sliders blend together, and on the right side, where the sinker and both changeup clusters sit almost on top of each other.





Velocity vs spin chart. On this chart, the groups that overlapped in movement separate. Sinkers sit out around 94 mph, the run heavy changeup sits around 87 mph and 2,000 rpm, and the low spin changeup and splitter group drops to around 1,400 rpm. The reverse is also true. Four seamers and sinkers overlap here but were clearly

apart on the movement chart, and curveballs and sweepers overlap here but sit in different spots on the movement chart.



Discussion

The biggest takeaway is that four simple measurements were enough for kmeans to put 93.6% of pitches into the right family without ever seeing a label, compared to a 49.1% floor. Both the elbow method and the silhouette width pointed to k = 3, which means fastballs, breaking balls, and offspeed pitches really are the natural groups in the data. The key ingredient turned out to be spin rate paired with velocity. That makes sense from a pitching standpoint. Fastballs are the hardest pitches with medium spin, breaking balls are slower with the most spin, and offspeed pitches are slower with the least spin, since killing spin is the whole idea behind a changeup or splitter.

Another takeaway is that the right combination of predictors mattered more than the number of them. All four together scored best, but adding a predictor didn't always help. Velocity plus horizontal break scored 89.5%, but adding vertical break dropped it to 80.0%, and only adding spin rate on top brought it back up. Movement on its own was weak because pitches from different families can move the same way. A changeup and a sinker both run arm side without much ride, and a cutter and a slider both move glove side. Velocity and spin rate are what pull them apart, which the velocity vs spin chart shows clearly.

Most of the misses came from cutters, and I think that's the most interesting finding. Statcast files every cutter under fastballs, but about 41% of them looked more like breaking balls to the model. That fits how cutters are actually thrown. Some pitchers throw a firm cutter that plays like a fastball with a little cut, while others throw something closer to a hard, short slider. So the cutter isn't really one pitch, it's a range that sits between the fastball and the slider. The other misses made sense too. The sinkers grouped with offspeed pitches are probably softer sinkers with a lot of run, the

changeups grouped with fastballs are probably firm changeups, and the sliders grouped with fastballs are likely hard gyro sliders that barely move.

When I asked for eight groups, kmeans found eight, but not Baseball Savant's eight. It supported treating the sweeper as its own pitch, since sweepers got their own cluster even when they were scored as sliders at k = 7. On the other hand, it split changeups into two styles separated mainly by about 560 rpm of spin, and it put most splitters inside the low spin changeup group instead of giving them their own cluster. I think this comes down to the difference between grip and shape. Some pitch labels describe how a pitch moves, while others describe how it's held. A splitter is a different grip than a changeup, and a knuckle curve is a different grip than a curveball, but they can produce the same shape. The model can only see the shape, so labels that are really about grip can't be recovered this way. The lower silhouette widths at k = 7 and k = 8 also fit this, since the specific pitch types blend into each other much more than the three families do.

There are a few limitations worth mentioning. Each row is a pitcher's average for a pitch type, not individual pitches, so it hides how much a pitch varies from one throw to the next. The accuracy is also measured against Statcast's labels, which aren't perfect themselves, so some of the "misses" may be pitches that are labeled questionably rather than mistakes by the model. Each row also counts the same no matter how many pitches it's based on, so a pitch thrown 60 times has the same pull on the clusters as one thrown 1,000 times. Splitters were a small group at only 82 rows, which made it harder for them to form a cluster of their own. The scoring method also has a blind spot at k = 7 and k = 8. Because each cluster is credited with its most common label, two clusters can both count as changeups while splitters never count as correct, so the accuracy shows how pure the clusters are, not whether every pitch type got a group of its own. Finally, kmeans tends to look for round groups of similar size, which isn't a great fit for something like the cutter, which stretches between two families.

For next steps, I would try clustering individual pitches for a single pitcher to see if his pitches form clean groups or if some of them blend together, which could point to a pitch that isn't consistent. I would also add arm angle and release point, since a pitch can move differently depending on the arm slot it comes from, and that could help separate groups that overlap here, like the cutters and hard sliders. Finally, I would try a clustering method that doesn't assume round groups, like a Gaussian mixture model, to see if it handles the cutter and the two changeup styles better.